МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине

«Операционные системы»

«Copy-on-write fork»

Работу выполнил:
Студенты группы P3411
Болорболд Аригуун

Лектор:
Клименков Сергей Викторович
Практик:
Смирнов Виктор Игоревич

г. Санкт-Петербург
2025

Содержание

1. Текст задания 3
   1.1. Часть 1. UB-on-write 3
   1.2. Часть 2. Профессиональные средства отладки 3
   1.3. Часть 3. Fault-on-write 4
   1.4. Часть 4. Copy-on-write 4
   1.5. Часть 5*. Ленивая аллокация 5
2. Ход работы 7
   2.1. Часть 1. UB-on-write 7
   2.2. Часть 2. Профессиональные средства отладки 7
   2.3. Часть 3. Fault-on-write 9
   2.4. Часть 4. Copy-on-write 11
   2.5. Часть 5*. Ленивая аллокация 14
3. Вывод 16

1. Текст задания

Системный вызов `fork` в текущей реализации копирует всю память родительского процесса в дочерний. Однако, как показывает практика, эта операция неэффективна: как правило, ни родительский, ни дочерний процесс не редактируют большую часть скопированной памяти, поэтому они могли бы пользоваться одной общей областью физической памяти.
Ещё хуже происходит в самом типичном сценарии использования `fork`:

```
int status = fork();
assert(status == 0);

const char *child_argv[] = {"/bin/some/program", "--help", NULL};
const char *child_envp[] = {NULL};
execve(child_argv[0], child_argv, child_envp);
```

Все страницы дочернего процесса незамедлительно уничтожаются. Кроме этого, сам процесс копирования занимает продолжительное время. Однако, совсем без копирования не обойтись — его можно делать в том случае, если один из процессов пытается что-то записать в область памяти. Такая оптимизация называется `copy-on-write`.
Наша цель — реализовать `copy-on-write` для системного вызова `fork`.

1.1. Часть 1. UB-on-write

Начнём с простого шага: уберём копирование памяти. Системный вызов `fork` (kernel-space код находится в `/kernel/proc.c#L244`) для этого вызывает функцию `uvmcopy` (`kernel/vm.c#L320`). Давайте упростим её — просто запишем в новую таблицу страниц те же физические адреса, что и в старую.
Как вы думаете, что пойдет не так? Попробуйте запустить обновленную версию и проверьте ваши предположения.

1.2. Часть 2. Профессиональные средства отладки

Напишите функцию `void vmprint(pagetable_t)` в `kernel/vm.c`, которая будет печатать содержимое таблицы страниц.
Вывод должен быть отформатирован следующим образом:

```
page table 0x0000000087f6e000
..0: pte 0x0000000021fda801 pa 0x0000000087f6a000
.. ..0: pte 0x0000000021fda401 pa 0x0000000087f69000
.. .. ..0: pte 0x0000000021fdac1f pa 0x0000000087f6b000
.. .. ..1: pte 0x0000000021fda00f pa 0x0000000087f68000
.. .. ..2: pte 0x0000000021fd9c1f pa 0x0000000087f67000
..255: pte 0x0000000021fdb401 pa 0x0000000087f6d000
```

```
.. ..511: pte 0x0000000021fdb001 pa 0x0000000087f6c000
.. .. ..510: pte 0x0000000021fdd807 pa 0x0000000087f76000
.. .. ..511: pte 0x000000002000200b pa 0x0000000080008000
```

Многоточиями (`..`) обозначаются уровни вложенности, в каждой строчке показывается индекс записи, PTE и физический адрес записи.
Чтобы не делать всё с нуля, воспользуйтесь макросами в конце файла `/kernel/riscv.h`, посмотрите на функцию `freewalk` (`/kernel/vm.c#L284`).
Не забудьте добавить определение функции в `kernel/defs.h`, чтобы воспользоваться ей в нужных частях ядра.
Вызовите эту функцию в `exec.c` и выведите таблицу страницу для первого процесса. Обратите внимание: для всех остальных процессов выводить таблицу не нужно.

1.3. Часть 3. Fault-on-write

Для реализации оптимизации воспользуемся следующим трюком: запретим и родительскому, и дочернему процессу писать в продублированные страницы. Если они всё же попробуют это сделать, процессор сгенерирует отказ страницы, который обрабатывается кодом ядра.
На этом шаге измените флаги `PTE` и запретите запись в страницы дочернего процесса, удалив флаг `PTE_W`. Перед запуском попробуйте представить, что произойдет. Запустится ли вот такая программа из шелла?

```
#include "user/user.h"

int main() {
    exit(0);
}
```

Попробуйте запустить ОС и `cowtest` в ней.

1.4. Часть 4. Copy-on-write

Переходим к самой важной части — копированию. Давайте теперь запретим запись как в страницы родительского, так и дочернего процесса в uvmcopy. Назовём такие страницы заблокированными.

Теперь при каждом обращении на запись в заблокированную страницу будет происходить отказ страницы. Мы можем обработать его в функции `usertrap` (`kernel/trap.c#L36`) — из всех ошибок нас интересует только отказ страницы — его код (`r_scause()`) равен 13 или 15. Обратите внимание на аргументы `printf` (`kernel/trap.c#L71`). Вам нужен регистр

`stval` (`r_stval()`) — в нём лежит виртуальный адрес, по которому произошла ошибка.

Возьмите код из старой версии `uvmcopy` и таким же образом скопируйте заблокированную страницу, после чего отобразите старые виртуальные адреса в новые физические. Не забудьте разрешить запись в новую страницу.

- Используйте макрос `PGROUNDDOWN` — вам нужен не адрес с ошибкой, а адрес страницы.
- Если что-то падает, поищите значение `sepc` в `kernel/kernel.asm` — оно отвечает за адрес инструкции, на которой произошла ошибка. Не бойтесь, с ассемблером разбираться не придётся: вместе с инструкциями там есть строки вашего исходного кода.

Вам, возможно, понадобится информация о том, какие страницы заблокированы. Для этого можно использовать биты `RSW` в `RISC-V PTE` — восьмой и девятый.

Освобождайте физические страницы в `/kernel/kalloc.c` только тогда, когда они никем не используются.

- Вам понадобится хранить счётчик ссылок. Делать это с помощью массива фиксированной длины (похожего на тот, от которого мы избавлялись в прошлом задании) — один из подходящих вариантов.

Помните, что если заблокированную страницу использует только один процесс, то её нужно просто разблокировать — так вы избежите утечки памяти.

Последний штрих — используйте ту же идею в `copyout` (`kernel/vm.c#L365`).

- В конце файла `kernel/riscv.h` вы найдете полезные макросы.

Убедитесь, что `cowtest` и `usertests` проходят. Первый тест из `cowtest`, к примеру, аллоцирует больше половины доступной памяти и делает `fork`.

- Вы можете убивать процесс, если при page fault на copy-on-write-странице в памяти не оказалось места.

1.5. Часть 5*. Ленивая аллокация

Программы хранят различные данные, как правило, в двух местах — в куче и на стеке. Чтобы получить место в куче, используется системный

вызов `sbrk`. Он увеличивает место, выделенное для программы, на указанное количество байт.

Нередко программы аллоцируют память «на всякий случай», фактически её не используя. Некоторые операционые системы обрабатывают аллокации лениво — лишь запоминают новое адресное пространство, выделяя страницы только при необходимости. В качестве дополнительного задания нам необходимо добавить эту возможность в `Xv6`.

1.5.1. Уберите аллокацию

Измените `sbrk` (`kernel/sysproc.c#L42`) так, чтобы он не выделял память. Это не сложнее, чем подзадание 1.

Попробуйте угадать, что произойдет при запуске системы, а затем проверьте ваше предположение.

1.5.2. Верните аллокацию

В этом задании снова нужно изменять `usertrap` (`kernel/trap.c#L36`).

В этот раз вам понадобится код из `uvmmalloc()`(`kernel/vm.c#L241`) — а именно вызов функций `kalloc()` и `mappages()`, которые вызывались из `sbrk()` через `growproc()`.

Не забудьте модифицировать `uvmunmap()` (`kernel/vm.c#L181`) и не освобождать ту память, которая не была фактически выделена.

Помимо этого, вам нужно будет обработать некоторое количество граничных случаев:

- Отрицательные аргументы `sbrk`
- Page-fault на неаллоцированной странице при нехватке памяти, а также на ещё не запрошенной странице (процесс тоже нужно убивать)
- Проверить, что копирование памяти в `fork()` работает корректно
- Корректно обрабатывать передачу ещё не выделенных адресов в системные вызовы — например, `read`. Всё это проверяется с помощью `lazytests` и `usertests`.
- Скорее всего, у вас не пройдут тесты `sbrkbasic` и `sbrkfail`. Посмотрите на функцию `wait` (`kernel/proc.c#L384`) и подумайте, как это исправить.

2. Ход работы

2.1. Часть 1. UB-on-write

```
int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
  pte_t *pte, *pte_new;
  uint64 i;
  for(i = 0; i < sz; i += PGSIZE){
    if((pte = walk(old, i, 0)) == 0)
      panic("uvmcopy: pte should exist");
    if((*pte & PTE_V) == 0)
      panic("uvmcopy: page not present");
    if ((pte_new = walk(new, i, 1)) == 0)
      panic("uvmcopy: pte copy failed");
    *pte_new = *pte;
  }
  return 0;
}
```

Вместо старого `uvmcopy`, который выделял новые страницы (`kalloc`) и копировал данные, мы сделали простое копирование на `pte_new` через `walk` — т.е. в новой таблице начал отображаться тот же самый физический адрес `pa`, который использует родитель. На уровне `PTE` это означает: теперь у родителя и у ребёнка есть `PTE`, указывающие на одну и ту же физическую рамку.
При запуске `fork` получаю следующее сообщение:

```
$ fork
exec fork failed
usertrap(): unexpected scause 0x2 pid=2
            sepc=0x0 stval=0x9000
scause=0xf sepc=0x800064a0 stval=0x0
panic: kerneltrap
```

Это указывает на то, что наша оптимизация привела к разделению физической памяти без реестра владения и без защиты записи, поэтому при `exec`/`exit` одна из сторон освобождает физическую страницу, а другая остаётся с висящей ссылкой — это `use-after-free` / `double-free`, что и вызывает падение ядра/неопределённое поведение. Такая оптимизация не сработает без соответствующих реализаций, о чем будет раздел дальше.

2.2. Часть 2. Профессиональные средства отладки

Сначала сделаем так, чтобы `vmprint` вызывался сразу при запуске.

```c
int
exec(char *path, char **argv)
  // ...
  if (p->pid) {
    vmprint(p->pagetable);
  }
  // ...
}
```

Потом непосредственно реализуем метод в `vm.c`:

```c
static void
vmprint_impl(pagetable_t pagetable, int depth)
{
  for(int i = 0; i < 512; i++){
    pte_t pte = pagetable[i];
    if(pte & PTE_V){
      for(int i = 0; i < depth; i++) {
        printf(".. ");
      }
      printf("%d: pte %p pa %p\n", i, (void*)pte,
(void*)PTE2PA(pte));
    }
    if((pte & PTE_V) && (pte & (PTE_R|PTE_W|PTE_X)) == 0){
      vmprint_impl((pagetable_t)PTE2PA(pte), depth + 1);
    } else if(pte & PTE_V){
      if (depth != 3) {
        panic("vmprint: leaf");
      }
    }
  }
}

void
vmprint(pagetable_t pagetable)
{
  printf("page table: %p\n", pagetable);
  vmprint_impl(pagetable, 1);
}
```

В итоге получаем:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
page table: 0x0000000087f4e000
.. 0: pte 0x0000000021fd2801 pa 0x0000000087f4a000
```

```
.. .. 0: pte 0x0000000021fd2401 pa 0x0000000087f49000
.. .. .. 0: pte 0x0000000021fd2c1b pa 0x0000000087f4b000
.. .. .. 1: pte 0x0000000021fd2017 pa 0x0000000087f48000
.. .. .. 2: pte 0x0000000021fd1c07 pa 0x0000000087f47000
.. .. .. 3: pte 0x0000000021fd1817 pa 0x0000000087f46000
.. 255: pte 0x0000000021fd3401 pa 0x0000000087f4d000
.. .. 511: pte 0x0000000021fd3001 pa 0x0000000087f4c000
.. .. .. 510: pte 0x0000000021fd5807 pa 0x0000000087f56000
.. .. .. 511: pte 0x0000000020001c0b pa 0x0000000080007000
init: starting sh
$
```

2.3. Часть 3. Fault-on-write

Так как теперь у родителя и у ребёнка есть `PTE`, указывающие на одну и ту же физическую рамку, я изначально ожидал, что ядро сразу впадёт в панику. Но оказалось, что нет. Это происходит из-за 2 причин, которые должны быть обеспечены одновременно:

- Есть учёт ссылок на физрамки (refcount): при создании ещё одной ссылки инкрементируется счётчик, и рамка не будет освобождена, пока счётчик не станет нулём.
- Страницы, которые совместно используются, делаются read-only (или в PTE снимается бит записи) и реализуется механизм Copy-On-Write: при попытке записи аппарат генерирует исключение, ядро обрабатывает trap, выделяет новую страницу, копирует данные и обновляет PTE (и refcounts).

Но это уже заходит в территорию следующего раздела. Давайте сначала реализуем этот пункт, в результате которого и родителю, и ребёнку запрещается права записи в память. Это реализуется путём снятия бита перед маппингом страницы:

```
// ...
    if ((pte_new = walk(new, i, 1)) == 0)
      panic("uvmcopy: pte copy failed");
    *pte_new = *pte;
    *pte_new &= ~PTE_W;
  }
// ...
```

где `PTE_L` — это (`riscv.h`):

```
#define PTE_L (1L << 8) // COW
```

который должно запретить запись в память. Этим реализуется уже первая часть чего мы хотим добиваться — Write. Если теперь запустить `cowtest`, то видим следующую картину.

```
$ cowtest
usertrap(): unexpected scause 0xf pid=3
            sepc=0x854 stval=0x4f98
usertrap(): unexpected scause 0x2 pid=2
            sepc=0x0 stval=0x9000
scause=0xf sepc=0x800065a0 stval=0x0
panic: kerneltrap
```

Первая строка вывода указывает на то, что при попытке записи вызвался `pagefault`, что является ожидаемым поведением. Так как у нас не реализован обработчик `Copy-on-Write`, ядро считает такое поведение «неожиданным» и падает в панику. Сама реализация есть на следующем разделе.

Почти забыл про пользовательскую программу. Вопрос: запустится ли программа следующего вида?

```
#include "kernel/types.h"
#include "user/user.h"

int main() {
    exit(0);
}
```

Короткий ответ: да, но в нашем экономике — нет.
Что ожидается в чистом xv6:

1. Шелл вызывает `fork`;
2. В новом дочернем процессе шелл вызывает `execve`, который читает ELF-файл, создает новое адресное пространство, потом загружает сегменты, готовит стек с `argv`/`argp`, устанавливает точку входа;
3. После успешного выполнения `exec` `CPU`/`hart` начинает выполнение кода — `main()`;
4. В `main()` выполняется `exit(0)`, который является оберткой системного вызова `SYS_exit`, тем самым завершив программу.

Программа выполнится (хотя ничего мы не заметим). Но в нашем условии (после прошлых 3 шагов) обычно бывают следующие проблемы:

- Шаг с UB-on-write. Так как мы маппим те же страницы без соответствующей механизмы обработки, то `fork`/`execve` могут привести к `double-free`/`use-after-free` и следовательно паник ядра. Тогда программа не запустится.

- Текущий шаг. Запись в таком сценарий приведёт к `pagefault` при попытке записи, и так как у нас нет обработчика, дальше приведет к `unexpected scause` — значит процесс будет убит или упадёт ядро. Но есть нюанс: для именно нашего случая записи в памяти нет, поэтому сама цепочка действии exec → run → exit не должна по идее вызвать ошибку. Однако сам вызов к `fork` может уже вызвать цепочку действия, которое ведет к более раннему проявлению неопределенности.

При запуске xv6 и последующем выполнении программы `badger`, система завершилась с ошибкой, а именно:

```
$ badger
usertrap(): unexpected scause 0xf pid=3
            sepc=0x854 stval=0x4f98
usertrap(): unexpected scause 0x2 pid=2
            sepc=0x1000 stval=0x2000
usertrap(): unexpected scause 0xc pid=1
            sepc=0x1000 stval=0x1000
panic: init exiting
```

2.4. Часть 4. Copy-on-write

Здесь будет реализована вторая часть нашей цели — Copy.

1. Сначала реализуем учёт ссылок на физическую страницу в `kalloc.c`. Это решит проблему с «висящими ссылками», которые приводят к неопределенному поведению в ядре.

```
void
kincref(void *pa)
{
  __sync_add_and_fetch(&kmem.refs[((char*)pa -
(char*)kmem.pa_start) / PGSIZE], 1);
}

void
kdecref(void *pa)
{
  __sync_sub_and_fetch(&kmem.refs[((char*)pa -
(char*)kmem.pa_start) / PGSIZE], 1);
}

uint
kgetrefs(void *pa)
{
  return kmem.refs[((char*)pa - (char*)kmem.pa_start) / PGSIZE];
}
```

```
// Использование этих методов в репозиторий в этом же файле
```

2. Добавим корректную обработку случаев, когда возвращается `pagefault` в `trap.c`. Сначала определим сверху файла коды ошибок, возвращаемый `r_scause()`. Они соответствуют ошибке при загрузке (13) и при записи (15).

```
#define LOAD_PAGE_FAULT 13 // page fault during load
#define AMO_PAGE_FAULT  15 // page fault during store
```

Потом обрабатываем эти случаи. `kill_process()` добавлен для того, чтобы не допускать использование `goto`.

```
static void
kill_process(struct proc *p, char *cause, uint64 scause, uint64
stval, uint64 sepc)
{
  printf("usertrap(): %s scause 0x%lx pid=%d\n", cause, scause, p-
>pid);
  printf("            sepc=0x%lx stval=0x%lx\n", sepc, stval);
  setkilled(p);
}

void
usertrap(void)
{
  // ...
      syscall();
  } else if((which_dev = devintr()) != 0){
    // ok
  } else if (scause == LOAD_PAGE_FAULT) {
    kill_process(p, "Load page fault", scause, stval, sepc);
  } else if (scause == AMO_PAGE_FAULT) {
    if(uvmcow(p->pagetable, r_stval()) != 0) {
      kill_process(p, "Store/AMO page fault", scause, stval, sepc);
    }
  } else {
    kill_process(p, "unexpected", scause, stval, sepc);
  }
  // ...
}
```

3. Реализуем `Copy-on-Write` в `vm.c`.

```
int
uvmcow(pagetable_t pagetable, uint64 va)
{
```

```
  pte_t *pte;
  va = PGROUNDDOWN(va);

  if(va >= MAXVA) {
    return -1;
  }
  if(!(pte = walk(pagetable, va, 1)))
    return -1;
  if(!((*pte & PTE_V) && (*pte & PTE_L)))
    return -1;

  uint64 pa = PTE2PA(*pte);
  if(kgetrefs((void*)pa) == 1) {
    *pte |= PTE_W;
    return 0;
  }

  char *mem = kalloc();
  if(mem == 0)
    return -1;

  memmove(mem, (char*)pa, PGSIZE);
  *pte = PA2PTE(mem) | PTE_FLAGS(*pte) | PTE_W;

  kdecref((void*)pa);
  return 0;
}
// Использования функции есть в репозитории
```

По идее это всё, что от нас требовало задание. Если провести проверку путем запуска теста `cowtests`:

```
$ cowtest
simple: ok
simple: ok
three: ok
three: ok
three: ok
file: ok
ALL COW TESTS PASSED
```

Также наша мини-программа теперь запускается без проблем.

```
$ badger
$
```

2.5. Часть 5*. Ленивая аллокация

2.5.1. Убрать аллокацию

```
uint64
sys_sbrk(void)
{
  uint64 addr;
  int n;
  struct proc *p = myproc();

  argint(0, &n);
  addr = p->sz;

  if (n > 0) {
    uint64 newsz;
    if((newsz = uvmalloc(p->pagetable, addr, addr + n, PTE_W)) ==
0) {
      return -1;
    }
    p->sz = newsz;
  } else if(n < 0) {
    p->sz = uvmdealloc(p->pagetable, addr, addr + n);
  }
  return addr;
}
```

Понятно, что `sbrk()` вызывается только при вызове программ, использующих кучу. Значит, система должна запускаться без проблем, но при вызове любой пользовательской программы, который использует кучу, должна быть ошибка.

```
$ ls
usertrap(): Store/AMO page fault scause 0xf pid=3
            sepc=0x115e stval=0x5008
panic: uvmunmap: not mapped
```

Это то, чего я ожидал.

2.5.2. Верните аллокацию

Опять изменим `usertrap()`:

Потом реализуем саму ленивую аллокацию:

```
int
uvmlazyalloc(pagetable_t pagetable, uint64 va)
{
```

```
  uint64 page = PGROUNDDOWN(va);
  pte_t *pte;
  uint flags;

  if(page >= MAXVA)
    return -1;
  if((pte = walk(pagetable, va, 1)) == 0)
    return -1;

  if(*pte == 0)
    return -1;

  flags = PTE_FLAGS(*pte);
  if((*pte & PTE_V) != 0 || (flags & PTE_M) == 0)
    return -1;

  char *mem = kalloc();
  if(mem == 0)
    return -1;

  memset(mem, 0, PGSIZE);
  *pte = (PA2PTE(mem) | (flags & ~PTE_M) | PTE_V);
  sfence_vma();
  return 0;
}
// Использование также в репозиторий
```

С добавлением ленивой аллокации пришлось модифицировать функции, которые также можно будет посмотреть в репозитории. Теперь связанные тесты проходят:

```
$ lazytests
lazytests starting
running test lazy alloc
sbrk() failed
test lazy alloc: OK
running test lazy unmap
sbrk() failed
test lazy unmap: OK
running test out of memory
test out of memory: OK
ALL TESTS PASSED
```

Но падает `usertests`:

```
test sbrkfail: usertrap(): Load page fault scause 0xd pid=6639
                                   sepc=0x3a52 stval=0x7ce2000
```

```
FAILED
SOME TESTS FAILED
```

Ситуация такая: в процессе А делается форк, получается дочерний процесс Б. (Оба процесса имеют cow память). Далее процесс Б занимает и аллоцирует всю имеющуюся память и выходит, пытаясь записать код возврата в память процесса А, которая полностью заблокирована на запись. Операция проваливается, так как код возврата не получается записать в заблокированную память (аллоцировать новую физ. память не получается так как она вся аллоцирована процессом Б).

Это исправляется через добавления `freeproc()` в функции `wait()`:

```
// ...
        havekids = 1;
        if(pp->state == ZOMBIE){
          // Found one.
          pid = pp->pid;
          freeproc(pp); // Здесь!
          if(addr != 0 && copyout(p->pagetable, addr, (char *)&pp-
>xstate,
                                  sizeof(pp->xstate)) < 0) {
            release(&pp->lock);
// ...
```

После чего все тесты начали проходить.

3. Вывод

В ходе выполнения работы был изучен такой метод оптимизации аллокации памяти как Copy-on-Write и Lazy, который реализуется через PTE флагов и модификации обработчика прерывании `trap`. Этим обеспечивается общее использование памяти до момента записи и выделение памяти только когда необходимо.